Good. I'll give you a clean, minimal step-by-step version.

We will:

- Simulate HR login
- Trigger UPDATE
- Verify audit entry
- Roll everything back

Nothing permanent will change.

🟢 Step 1 — Open PostgreSQL

```
sudo -iu postgres
psql
\c corporate_db
```

You should see:

```
corporate_db=#
```

🟢 Step 2 — Simulate Logged-in User (HR)

```
SET app.username = 'hr1';
SET app.role = 'HR';
SET app.department = 'HR';
SET app.location = 'Internal';
```

This is required because your trigger reads **`app.username`**.

🟢 Step 3 — Start Safe Transaction

```
BEGIN;
```

From now on, everything is temporary until **`ROLLBACK`**.

🟢 Step 4 — Perform an Update

```
UPDATE employees
SET salary = salary + 1000
WHERE department = 'HR';
```

Trigger should fire automatically.

🟢 Step 5 — Check Audit Table

```
SELECT *
FROM audit_logs
ORDER BY audit_id DESC
LIMIT 5;
```

You should see a new row with:

- user_id → hr1's ID
- action → UPDATE
- table_name → employees
- result → SUCCESS

If you see that → audit works.

🟢 Step 6 — Revert Everything

```
ROLLBACK;
```

Now:

✔️ salaries return to original
✔️ audit row disappears
✔️ database state is clean

🟢 Final Check (Optional)

```
SELECT * FROM audit_logs ORDER BY audit_id DESC LIMIT 5;
```

The temporary audit row should be gone.

That's it.

Trigger works if:

- UPDATE succeeds
- Audit row appears before ROLLBACK
- Everything disappears after ROLLBACK

Tell me what you observe after Step 5.

sudo \-iu postgres
but here i was logged in as postgres user, and not app hr

Excellent catch 👋 — very important distinction.

You are absolutely right:

When you do:

```
sudo -iu postgres
psql
```

You are:

- Linux user → postgres
- Database user → postgres (DB superuser)

So yes — you are NOT actually logged in as `app_hr`.

🧠 So How Is It Still Acting Like HR?

Because of this:

```
SET app.username = 'hr1';
SET app.role = 'HR';
SET app.department = 'HR';
```

Your trigger does **NOT** use:

```
current_user
```

It uses:

```
current_setting('app.username')
```

That means:

The trigger trusts the session variable,
NOT the database login role.

So even though you are DB superuser,
you're simulating application login via session context.

🔍 Two Separate Things Happening

There are two identity layers:

1️⃣ PostgreSQL login role

Who connected to database (postgres / app_hr / etc.)

2️⃣ Application context variables

Who the application says the logged-in user is

Your design is built around layer 2.

This is realistic.

Most real systems:

- Connect as one DB service account
- Inject session context per request
- Enforce RLS using that context

Exactly like you're doing.

🚨 BUT There Is One Important Caveat

When you connect as:

```
postgres
```

Postgres superuser has:

```
Bypass RLS
```

So:

If you test SELECT policies as postgres,
Row-Level Security will NOT be enforced.

But for testing triggers (UPDATE auditing),
this is fine.

🎯 If You Want to Test As Real app_hr

You can instead do:

```
psql -U app_hr -d corporate_db
```

Then set:

```
SET app.username = 'hr1';
SET app.role = 'HR';
SET app.department = 'HR';
```

That is closer to production behavior.

🧱 What Should You Use For Testing?

For audit trigger:
✔️ Can use postgres
✔️ Because trigger doesn't depend on RLS enforcement

For RLS testing:
❌ Do NOT use postgres
✔️ Use app_hr, app_employee, etc.

🔥 Clean Testing Strategy

For trigger logic:

Use postgres + SET variables.

For RLS behavior:

Login as real app role.

🧠 Final Clarification

You were not “wrong.”

You were:

- DB superuser
- But simulating application user via context

That’s why audit worked.

Now tell me:

Do you want to test:

1️⃣ Trigger behavior only
or
2️⃣ Full real login as app_hr with RLS enforcement

We’ll do it properly.